

Terraform Exam

חניל זרבאילוב Hanil Zarbailov

319331419

Section 1: Q&A (20 Questions)

- 1) Terraform is an Infrastructure As Code tool that enables the user to configure and deploy entire cloud environments in a few minutes.
- 2) The Terraform language is declarative, describing an intended goal, rather than the steps to reach that goal. It manages its state by comparing what is already available on the environment, what needs changing and what's missing.
- 3) Terraform providers are plugins that enable the user to interact with cloud providers, SaaS providers, and other APIs.
- 4) Terraform creates resources in parallel and can automatically handle resource dependencies in most cases using implicit dependencies. If not specified, Terraform can't know if a resource must be created before or after another one if there is no clear relationship within the Terraform code itself.
- 5) Providers, resources and outputs.
- 6) terraform refresh - refreshes the state based on the remote state, terraform plan - show the user what will get updated in the environment without applying

the changes, terraform apply - confirm and apply the changes to the remote environment.

- 7) Local backend runs in the current machine where terraform runs, Remote backend runs in the remote machine that gets created.
- 8) Enabling State Locking helps prevent resource configuration conflicts or corruption, which running simultaneous Terraform commands might cause.
- 9) Terraform modules serve as blueprints of resource configuration or other entities for the user. The author can require the user to pass certain data fields in order to use the module.
- 10) To pass variables, you specify the variable name and the value. In the module itself you specify which variables are required
- 11) Count specifies that you want multiple resources of the same configuration. With for_each you can loop over the instances and make specific changes between them.
- 12) You need to provide the http link or ssh connection in order to source the module.
- 13) You need to specify the ami, instance type and provide the security group ids.
- 14) You need to specify the cidr block. For further configuration, you can create private and public subnets, internet gateway and route tables.
- 15) You need to specify the policy name, description and JSON. after that you can attach it to a user with "aws_iam_user_policy_attachment".

- 16)** You need to create a target group with a name, port, protocol and vpc_id, after that create an EC2 launch template with a name_prefix, image_id, instance_type and security group, after that create an auto scaling group by specifying min_size, max_size, desired_capacity, launch configuration and vpc_zone_id(subnets). After creating all of the above, create an “aws_lb” by specifying its’ name, internal(bool), load_balancer_type, security groups and subnets. Create an “aws_lb_listener” to route HTTP traffic to a target group. To finish up, attach the target group to the autoscaling group with “aws_autoscaling_attachment”.
- 17)** The terraform validate command validates the configuration files in a directory, referring only to the configuration and not accessing any remote services such as remote state, provider APIs, etc. Validate runs checks that verify whether a configuration is syntactically valid and internally consistent, regardless of any provided variables or existing state.
- 18)** You can create mocks of resources or variables.
- 19)** The ignore_changes feature is intended to be used when a resource is created with references to data that may change in the future, but should not affect said resource after its creation.
- 20)** You can query for existing remote resources by using “data”.